Transformer-Koopman: History-Conditioned Low-Rank Transition Modulation with Rollout-Aware Training and Real-Time Predictive Control

Bolin Tao

***Abstract*—Koopman modeling represents nonlinear dynamics as linear evolution in a lifted space whose accuracy depends on the lifting map and the identified operator. Many conventional EDMD and learned-lifting approaches use instantaneous-state observables with a fixed lifted transition, so they neglect short-term temporal correlations and cannot distinguish how the system evolves from different past trajectories. History-aware lifting recovers that information, but one global lifted transition still forces all history-dependent regimes to share a single evolution law. We propose Transformer-Koopman, where a patch-based Transformer encoder maps a finite history window to a structured lifted state and a low-rank history-dependent term modulates a full-rank nominal transition. We show that a one-step-trained history-conditioned operator develops a training–deployment inconsistency under recursive scheduled propagation that spectral stabilization does not repair, whereas training under the deployment rollout restores long-horizon accuracy. The predictor is integrated into a one-iteration real-time-iteration (RTI) MPC that avoids re-running the full encoder at every stage, achieving a $p95$ latency of 120 ms under a 200-ms sampling period and tracking a matched exact scheduled NMPC oracle within 1%. A finite-horizon theory characterizes the rollout-error propagation and the local RTI approximation, supported by empirical diagnostics. Modeling and real-time control results on a 2-state soft robotic platform support the reported claims; a 6-state manipulator and hardware validation are left for future work.**

***Index Terms*—Koopman operator, history-conditioned dynamics, low-rank transition, rollout-aware training, model predictive control, real-time iteration, temporal lifting, Transformer**

I. Introduction

Learning predictive models for nonlinear controlled systems is a central problem in system identification and model-based control [4]–[6]. For controller design, model quality is determined by prediction accuracy and by compatibility with control synthesis; models that are both accurate and structurally suited to control design are therefore of primary interest.

The Koopman framework addresses this by representing nonlinear dynamics as a linear evolution in a lifted space of observable functions [1], [16]. This linear structure enables standard linear prediction and controller design tools [17], [29], making Koopman modeling an attractive paradigm for control. Its effectiveness depends critically on the choice of lifting map.

Many conventional EDMD [8] and learned-lifting approaches [18], [20] use instantaneous-state observables together with a fixed lifted transition, neglecting short-term temporal correlations. Delay-embedding variants enrich the observables with time-shifted measurements [11], [12], and context-aware formulations regenerate the operator from recent data [14], [15]. Enriching the observables with history supplies temporal information that the current measurement alone does not, but it leaves a second question open: a single time-invariant lifted transition still requires every history-dependent operating regime to share one global linear evolution in the learned coordinates, which can place a strong burden on the lifting map when the effective dynamics vary with recent trajectory context. We therefore separate history dependence into two roles—history is encoded into the lifted state, and a structured low-rank correction selectively adjusts a shared nominal transition. Such distinctions can matter more under recursive multi-step prediction.

We propose Transformer-Koopman, a temporal lifting framework with two components. The first component uses a Transformer encoder to map a finite history window into a structured lifted state that retains the measured physical state. The second identifies the Koopman dynamics from the lifted state: rather than requiring all history-dependent regimes to share a single time-invariant operator, we let the history modulate the lifted transition through a low-rank term, so that the transition can adapt to the recent trajectory while the propagation remains linear in the lifted state for a given context. We further show that a history-conditioned operator trained only on encoder-consistent one-step transitions develops a training–deployment inconsistency under recursive scheduled propagation—predicted latent states alter the predicted history, which in turn reschedules the subsequent transition operators—and that spectral stabilization alone does not repair this failure mode. We therefore train the predictor under the same recursively scheduled rollout used at deployment, which restores long-horizon accuracy.

Building on the rollout-trained scheduled predictor, we develop a real-time predictive controller based on a one-iteration RTI-MPC that avoids re-running the full Transformer lifting encoder at every prediction stage, and we establish a finite-horizon theory whose assumptions are assessed by empirical diagnostics on the validation distribution.

The contributions of this paper are summarized as follows:

1) **Low-rank history-conditioned modulation of the lifted transition.** We introduce a Koopman model in which finite-history information forms the lifted repre-

sentation and selectively modulates a full-rank nominal transition through a low-rank history-dependent update, while preserving linear lifted-state propagation for a given context, which is compatible with the scheduled RTI formulation developed later.

2) **Scheduled-rollout inconsistency and deployment-consistent training.** We identify a training–deployment mismatch that arises when a one-step-trained history-conditioned Koopman model is recursively propagated with its context rescheduled from predicted histories, and show that spectral stabilization alone does not repair this failure mode. Training under the same scheduled propagation used at deployment restores long-horizon prediction accuracy, to a level close to recurrent re-encoding.
3) **Real-time predictive deployment.** The rollout-trained scheduled predictor is integrated into a one-iteration RTI-MPC that avoids a full Transformer lifting encoder evaluation at every prediction stage. The resulting controller tracks a matched exact scheduled NMPC oracle within 1%, and a finite-horizon analysis supported by empirical diagnostics characterizes the rollout-error propagation and the local RTI approximation.

II. Related Work

A. History-Based Koopman Representation

Koopman operator theory represents nonlinear dynamics as a linear evolution on the observable functions of the state [1]–[3], enabling the direct application of linear control synthesis to nonlinear systems. Extensions such as DMDc [26], input-aware formulations [27], [28], and Koopman-based MPC [29] established the foundation for Koopman control, and later work added learned representations and performance guarantees [30]–[33]. To learn such models from data, EDMD fits observables from a pre-defined dictionary [8], [9], [34], whereas data-driven methods learn the representations instead [18]–[21]. Many conventional EDMD and learned-lifting approaches use instantaneous-state observables together with a fixed lifted transition, neglecting short-term temporal correlations and limiting long-horizon prediction accuracy.

Two lines relax this restriction. Delay-embedding variants enrich the dictionary with time-shifted measurements or Hankel/HAVOK coordinates [10]–[13], so the operator acts on a history-augmented state, but history enters only through the observables. Transformers model temporal dependencies through self-attention [35], while patch-based tokenization provides an efficient representation of local temporal structure [36]. In Koopman lifting, where the relevant temporal context is unknown a priori, Transformers can incorporate short-term information from past trajectories.

B. Context-Dependent Koopman Dynamics and Recursive Prediction

A history-augmented representation, however, still leaves the transition itself time-invariant. Context-aware and time-varying formulations instead let the effective operator depend on recent data: Koopa computes context-aware operators for non-stationary forecasting [14], and MamKO generates time-varying Koopman matrices from online history [15]. Our formulation separates history dependence into two roles: history is encoded into the lifted state, while a low-rank family identified jointly with the lifting map selectively corrects a shared nominal transition, so that the lifted dynamics remain linear in the lifted state for a given context and provide the conditional-linear structure exploited by the scheduled RTI controller.

Recent work has explored Transformer encoders with Koopman-style latent dynamics for forecasting [37], [38]. In particular, [38] parameterizes the transition as a single low-rank operator with a spectrally constrained diagonal factor on a patch-based Transformer backbone; unlike our full-rank nominal transition modulated by history, it uses a static low-rank transition for forecasting. In contrast, our Transformer is used only for history encoding, while prediction inside the controller is performed by the structured Koopman model. Relatedly, low-rank space-varying transitions have been learned for continuum-robot control, conditioned on the instantaneous state and paired with an infinite-horizon linear-quadratic controller [39].

Multi-step objectives have also been used in learned Koopman models to reduce recursive prediction error [20], [22]–[24], and are standard in neural system identification more broadly [25], so multi-step loss itself is not the focus here. The additional issue in history-conditioned dynamics is that recursive prediction reschedules future transitions from predicted histories, causing both the latent state and the conditioning context to depart from their one-step training distribution. Our rollout-aware training targets this specific deployment mismatch.

C. Real-Time Predictive Control

Two established strategies reduce the online cost of predictive control. The real-time iteration (RTI) performs a single sequential quadratic programming update per sampling instant around the current nominal trajectory [43], [44], and its nominal stability, suboptimality, and sensitivity-based delay reduction have been analyzed [45], [46]. Quasi-linear parameter-varying (quasi-LPV) control instead embeds the nonlinear model into a family of linear models indexed by a scheduling variable, so that freezing or iterating the scheduling trajectory yields a sequence of tractable quadratic programs [47]–[50].

Our controller inherits the one-iteration RTI structure but differs in where the scheduling signal comes from. Quasi-LPV MPC reads the scheduling variable from the state or velocity, using the previous solution or a planned reference; here the transition is scheduled by a context that a learned encoder extracts from a history window, and during prediction the history is itself produced by the predictor, so the scheduling trajectory is endogenous and cannot be read off the measured state. The context must therefore be re-evaluated along the rolled-out history rather than assumed, and the full Transformer lifting encoder need not be re-run at every prediction stage, letting the single-iteration controller fit inside the sampling period.

III. Preliminaries and Problem Formulation

In this section, we review the controlled Koopman formulation, introduce finite-history lifting, and state the data-driven learning problem considered in this work. The specific Transformer realization and the history-conditioned low-rank transition are developed in Section IV.

A. Koopman Modeling and Temporal Lifting

Consider a discrete-time controlled nonlinear system with physical state $\mathbf{s}_k \in \mathbb{R}^{n_s}$,

$$\mathbf{s}_{k+1} = f(\mathbf{s}_k, \mathbf{u}_k), \tag{1}$$

where $\mathbf{u}_k \in \mathbb{R}^m$ is the control input and f is an unknown nonlinear transition map. Only a measurement of the physical state is available,

$$\mathbf{x}_k = h(\mathbf{s}_k) \in \mathbb{R}^n, \tag{2}$$

with $h : \mathbb{R}^{n_s} \to \mathbb{R}^n$ an unknown measurement map. The predictor is restricted to the measured coordinates $\mathbf{x}_k$; when h is the identity and $n_s = n$, (1)–(2) reduce to the standard fully observed setting. When the measurement is not a complete state of (1), a deterministic first-order map $\mathbf{x}_{k+1} = F(\mathbf{x}_k, \mathbf{u}_k)$ need not exist, which is why finite history is introduced below. Measured state–input trajectories $\{(\mathbf{x}_k, \mathbf{u}_k)\}$ are assumed to be available for data-driven modeling.

The Koopman framework represents nonlinear dynamics through the evolution of observable functions rather than directly through the state [1], [3], [7], [16]. For a fixed control input $\mathbf{u}$, the input-parameterized Koopman operator $\mathcal{K}_\mathbf{u}$ acts on a scalar observable $\varphi : \mathbb{R}^{n_s} \to \mathbb{R}$ of the physical state as

$$(\mathcal{K}_\mathbf{u}\varphi)(\mathbf{s}) = \varphi(f(\mathbf{s}, \mathbf{u})). \tag{3}$$

For each fixed $\mathbf{u}$, $\mathcal{K}_\mathbf{u}$ is linear in the observable:

$$\mathcal{K}_\mathbf{u}(\alpha\varphi + \beta\psi) = \alpha\mathcal{K}_\mathbf{u}\varphi + \beta\mathcal{K}_\mathbf{u}\psi. \tag{4}$$

The Koopman operator is generally infinite-dimensional, so practical computation relies on a finite-dimensional approximation. In EDMD, one selects d scalar observables and stacks them into the lifting map

$$\mathbf{z}_k = g(\mathbf{x}_k) = [g_1(\mathbf{x}_k), \dots, g_d(\mathbf{x}_k)]^\top \in \mathbb{R}^d, \tag{5}$$

where each g_j is evaluated on the measurement $\mathbf{x}_k = h(\mathbf{s}_k)$, so the selected observables are of the form $g_j \circ h$. The lifted dynamics are then approximated by the input-affine linear system

$$\mathbf{z}_{k+1} \approx \mathbf{A}\mathbf{z}_k + \mathbf{B}\mathbf{u}_k, \tag{6}$$

where $\mathbf{A} \in \mathbb{R}^{d\times d}$ is the nominal lifted transition in the selected coordinates and $\mathbf{B} \in \mathbb{R}^{d\times m}$ maps the current input into the lifted space. Equation (6) is an input-affine finite-dimensional *surrogate* of the controlled observable evolution, adopted for prediction and control: for a fixed $\mathbf{u}$, $\mathcal{K}_\mathbf{u}$ acts on observables, whereas $(\mathbf{A}, \mathbf{B})$ parameterizes a linear recursion on a chosen d-dimensional dictionary, and $\mathbf{A}$ is not claimed to equal $\mathcal{K}_\mathbf{u}$. In this work the pair $(\mathbf{A}, \mathbf{B})$ is identified by the gradient-based procedure of Section IV rather than by a closed-form EDMD regression. Instantaneous-state lifting may miss temporal information that matters for prediction, particularly when the measurement does not by itself characterize the underlying dynamics.

To incorporate recent trajectory information, we construct the lifting map from a history window of length P:

$$\mathbf{X}_k = [\mathbf{x}_{k-P+1}, \dots, \mathbf{x}_k]^\top \in \mathbb{R}^{P\times n}, \tag{7}$$

and define

$$\mathbf{z}_k = g_\theta(\mathbf{X}_k) \in \mathbb{R}^d. \tag{8}$$

The resulting temporal lifting incorporates recent trajectory information into the latent state, and the corresponding nominal lifted dynamics are those of (6). In this work, g_θ is implemented using a Transformer encoder; the remaining question is how to accommodate history-dependent variations in the lifted dynamics without abandoning a control-compatible structure.

B. Learning Problem and Design Objectives

We assume that the available data consist of recorded state–input trajectories

$$\mathcal{D} = \{\tau^{(j)}\}_{j=1}^{N_{\mathrm{tr}}}, \qquad \tau^{(j)} = \{(\mathbf{x}_t^{(j)}, \mathbf{u}_t^{(j)})\}_{t=0}^{T_j}. \tag{9}$$

where N_{tr} denotes the number of recorded trajectories and T_j the number of samples in the j-th trajectory. For each admissible time index k, the state-history window $\mathbf{X}_k$ is defined by (7), and the aligned past-input history is

$$\mathbf{U}_k^{\mathrm{in}} = [\mathbf{u}_{k-P+1}, \dots, \mathbf{u}_{k-1}]^\top \in \mathbb{R}^{(P-1)\times m}. \tag{10}$$

The window therefore contains P measured states together with the $P-1$ completed input–state transitions that produced them, while the current input $\mathbf{u}_k$ acts separately through the input channel $\mathbf{B}\mathbf{u}_k$. Whenever the two histories enter a vector-valued map, they are flattened as $\operatorname{vec}\mathbf{X}_k \in \mathbb{R}^{Pn}$ and $\operatorname{vec}\mathbf{U}_k^{\mathrm{in}} \in \mathbb{R}^{(P-1)m}$.

Because the measurement need not form a complete Markov state, the predictor must exploit finite history. Encoding history only in the lifting map, however, still leaves a single global lifted transition shared by every history-dependent regime.

We therefore adopt three design objectives for the predictor: (i) exploit finite history to represent temporal information that the instantaneous measurement does not provide; (ii) let the lifted transition adapt to recent trajectory context while retaining, *for a fixed context*, an input-affine lifted propagation that provides the structure exploited by local linearization and real-time iteration—the scheduled finite-horizon problem itself remains generally nonlinear, because the context depends on the predicted history and hence on preceding control decisions (Section VI); and (iii) remain accurate when its own predicted histories are recursively fed back during multi-step prediction. These objectives specify what the predictor must achieve; they do not by themselves single out a unique model class. Section IV presents the state-preserving lifting and the history-conditioned low-rank transition adopted here to meet them, and Section V analyzes the finite-horizon behavior of that choice.

IV. History-Conditioned Transformer–Koopman Modeling

In this section, we instantiate the predictor of Section III-B using a patch-based temporal lifting, a history-conditioned low-rank transition, and deployment-consistent scheduled training.

A. Temporal Lifting and Low-Rank Transition

Fig. 1 summarizes the prediction pipeline: a patch-based Transformer encoder maps the history window to a history feature, the feature is concatenated with the current measurement to form the lifted state, and a context-conditioned lifted transition propagates that state forward.

Given the history window $\mathbf{X}_k \in \mathbb{R}^{P\times n}$, we partition it into $N_p = P/p$ contiguous non-overlapping temporal patches of length p, where p is assumed to divide P. The ith patch is denoted by

$$\mathbf{X}_k^{(i)} \in \mathbb{R}^{p\times n}, \qquad i = 1, \dots, N_p. \tag{11}$$

Each patch is vectorized as

$$\mathbf{q}_k^{(i)} = \mathrm{vec}(\mathbf{X}_k^{(i)}) \in \mathbb{R}^{pn}, \tag{12}$$

which is then mapped into the Transformer embedding space through a shared linear layer,

$$\boldsymbol{\phi}_k^{(i)} = \mathbf{W}_p \mathbf{q}_k^{(i)} + \mathbf{b}_p \in \mathbb{R}^{d_{\mathrm{model}}}, \tag{13}$$

where $\mathbf{W}_p \in \mathbb{R}^{d_{\mathrm{model}}\times pn}$ and $\mathbf{b}_p \in \mathbb{R}^{d_{\mathrm{model}}}$. This patch-based tokenization reduces the sequence length from P to $N_p = P/p$ while preserving local temporal structure within each patch.

The token sequence is augmented with sinusoidal positional encoding [35] to incorporate temporal order. With $\mathbf{p}^{(i)} \in \mathbb{R}^{d_{\mathrm{model}}}$ the positional encoding of the ith patch, the augmented tokens are

$$\tilde{\boldsymbol{\phi}}_k^{(i)} = \boldsymbol{\phi}_k^{(i)} + \mathbf{p}^{(i)}, \tag{14}$$

and the resulting token sequence is encoded by an L-layer Transformer encoder as

$$\mathbf{H}_k = \mathrm{TransformerEncoder}\Big(\{\tilde{\boldsymbol{\phi}}_k^{(i)}\}_{i=1}^{N_p}\Big) \in \mathbb{R}^{N_p\times d_{\mathrm{model}}}, \tag{15}$$

where L is the number of encoder layers and $\mathbf{H}_k^{(i)} \in \mathbb{R}^{d_{\mathrm{model}}}$ denotes the contextualized representation of the ith patch.

We aggregate the patch features by mean pooling to obtain a fixed-dimensional history descriptor:

$$\mathbf{r}_k = \frac{1}{N_p}\sum_{i=1}^{N_p} \mathbf{H}_k^{(i)} \in \mathbb{R}^{d_{\mathrm{model}}}. \tag{16}$$

After that, the pooled representation is projected to the history-feature space,

$$\mathbf{h}_k = \mathbf{W}_h \mathbf{r}_k + \mathbf{b}_h \in \mathbb{R}^{d-n}, \tag{17}$$

where $\mathbf{W}_h \in \mathbb{R}^{(d-n)\times d_{\mathrm{model}}}$ and $\mathbf{b}_h \in \mathbb{R}^{d-n}$.

The lifted state is defined as

$$\mathbf{z}_k = [\mathbf{x}_k;\, \mathbf{h}_k] \in \mathbb{R}^d, \tag{18}$$

where the first n components of $\mathbf{z}_k$ are the measured state $\mathbf{x}_k = h(\mathbf{s}_k)$ of Section III-A, and the remaining $d-n$ components correspond to the temporal features $\mathbf{h}_k$. This structure allows exact recovery of the *measured* coordinates through the fixed projection

$$\mathbf{x}_k = \mathbf{C}\mathbf{z}_k, \qquad \mathbf{C} = [\mathbf{I}_n\ \mathbf{0}], \tag{19}$$

it does not recover components of the physical state $\mathbf{s}_k$ that the measurement does not observe.

Although the temporal lifting provides different latent representations for different histories, a fixed transition $\mathbf{A}$ still requires all such history-dependent latent states to evolve under one global linear operator. To relax this additional restriction without discarding the conditional-linear structure, we decompose the transition into a shared nominal component and a structured history-dependent correction.

Definition 1 (History-Conditioned Low-Rank Koopman Family)**.** *Let the history window be* $\mathbf{X}_k$ (7)*, and let* $\mathbf{U}_k^{\mathrm{in}} \in \mathbb{R}^{(P-1)\times m}$ *denote the input history over the same window* (10)*. Define the lifted state and the context vector*

$$\begin{aligned} \mathbf{z}_k &= g_\theta(\mathbf{X}_k) = [\mathbf{x}_k;\, \mathbf{h}_k], \\ \mathbf{c}_k &= C_\phi(\mathbf{X}_k, \mathbf{U}_k^{\mathrm{in}}) \in \mathbb{R}^{d_c}, \end{aligned} \tag{20}$$

where C_ϕ *is a context network and, in the parameterization used here, the two histories of Section III-B are concatenated in flattened form into a single input vector of dimension* $Pn + (P-1)m$*, i.e.* $C_\phi(\mathbf{X}_k, \mathbf{U}_k^{\mathrm{in}}) \equiv \tilde{C}_\phi\big([\mathrm{vec}\,\mathbf{X}_k; \mathrm{vec}\,\mathbf{U}_k^{\mathrm{in}}]\big)$*. The lifting map uses the state history to construct a state-preserving representation, whereas the context map additionally uses the past inputs to condition the effective transition on how the current dynamical condition was reached. The model propagates the latent state by*

$$\hat{\mathbf{z}}_{k+1}^P = \mathbf{A}(\mathbf{c}_k)\,\mathbf{z}_k^E + \mathbf{B}\,\mathbf{u}_k, \qquad \mathbf{z}_k^E = g_\theta(\mathbf{X}_k), \tag{21}$$

with the history-conditioned *transition*

$$\mathbf{A}(\mathbf{c}) = \mathbf{A}_0 + \mathbf{U}\,\mathrm{diag}\big(\gamma_\psi(\mathbf{c})\big)\,\mathbf{V}^\top, \qquad \mathrm{rank}\big[\mathbf{A}(\mathbf{c}) - \mathbf{A}_0\big] \le r. \tag{22}$$

Here $\mathbf{A}_0 \in \mathbb{R}^{d\times d}$ *is the shared nominal transition,* $\mathbf{U}, \mathbf{V} \in \mathbb{R}^{d\times r}$ *are the low-rank factors,* $\gamma_\psi : \mathbb{R}^{d_c} \to \mathbb{R}^r$ *supplies the context-dependent modulation coefficients, and* $\mathbf{B} \in \mathbb{R}^{d\times m}$ *is the input matrix, which is* context-independent *(not context-modulated) and shared across all contexts. The integer* r *bounds the rank of the correction,* $\mathrm{rank}[\mathbf{A}(\mathbf{c}) - \mathbf{A}_0] \le r$*, and is a model hyperparameter whose value is reported with the experimental configuration.*

Three remarks follow. First, (21) is a prediction rule rather than a Koopman closure of the encoder: the model-propagated latent $\hat{\mathbf{z}}_{k+1}^P$ is not asserted to equal $g_\theta(\mathbf{X}_{k+1})$, and the two differ by the one-step residual (36), whose propagation under recursive prediction is analyzed in Section V-A. Second, context dependence is confined to $\mathbf{A}(\mathbf{c})$: the current input enters through the single global matrix $\mathbf{B}$, so for a fixed context the lifted propagation (21) is input-affine in $(\mathbf{z}_k, \mathbf{u}_k)$, which is the structure exploited by the local linearization of Section VI; over a multi-step prediction horizon the context depends on the predicted history and therefore on preceding control decisions, so the scheduled finite-horizon problem is generally nonlinear. Third, the low-rank term adds a bounded, structured correction to the nominal transition, and all parameters are optimized jointly with the temporal representation, as described next.

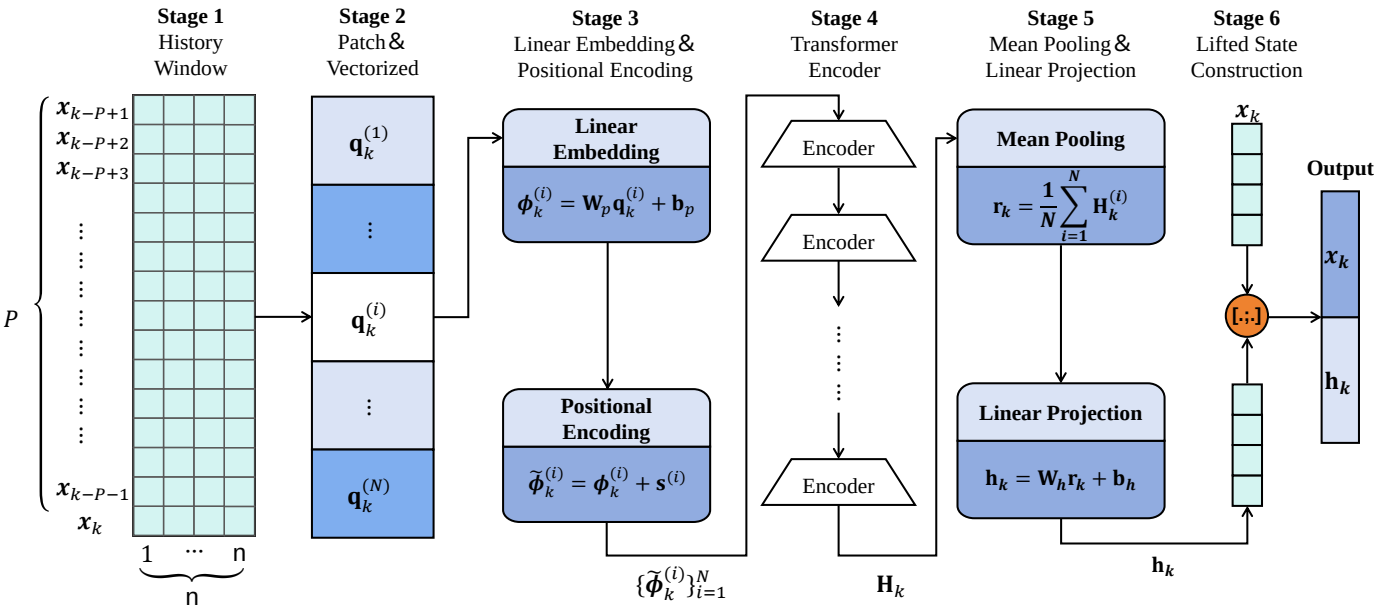


Fig. 1. Overall prediction pipeline of the proposed Transformer-Koopman framework. Given a history window $\mathbf{X}_k$ and the current control input $\mathbf{u}_k$, the Transformer encoder extracts a history feature $\mathbf{h}_k$, which is concatenated with the current state $\mathbf{x}_k$ to form the lifted state $\mathbf{z}_k = [\mathbf{x}_k; \mathbf{h}_k] \in \mathbb{R}^d$. The Koopman linear dynamics $\hat{\mathbf{z}}_{k+1} = \mathbf{A}(\mathbf{c}_k)\mathbf{z}_k + \mathbf{B}\mathbf{u}_k$ propagate the lifted state forward, and the predicted next physical state $\hat{\mathbf{x}}_{k+1} = \mathbf{C}\hat{\mathbf{z}}_{k+1}$ is recovered through a fixed linear projection.

B. Training Procedure

The model of Definition 1 has three trainable modules: the lifting encoder with parameters θ that produce the lifted state $\mathbf{z}_k$, the context network with parameters ϕ that produce $\mathbf{c}_k = C_\phi(\mathbf{X}_k, \mathbf{U}_k^{\text{in}})$, and the dynamics module, which comprises the nominal transition $\mathbf{A}_0$, the low-rank factors $\mathbf{U}, \mathbf{V}$, the modulation parameters ψ, and the input matrix $\mathbf{B}$. The input matrix $\mathbf{B}$ is context-independent—a single global matrix shared across all contexts—but it is optimized jointly with the other parameters rather than held frozen. The projection $\mathbf{C}$ is structurally fixed throughout training, as in Definition 1. Let H_R denote the training rollout horizon. Each training sample is a segment of H_R future steps cut from a single recorded trajectory, providing an aligned tuple: the history window $\mathbf{X}_k$, the past-input history $\mathbf{U}_k^{\text{in}}$, the future inputs $\mathbf{u}_k, \ldots, \mathbf{u}_{k+H_R-1}$, and the future measured states $\mathbf{x}_{k+1}, \ldots, \mathbf{x}_{k+H_R}$ used as prediction targets. Segments do not cross trajectory boundaries.

The history-conditioned transition introduces an additional deployment issue: during one-step training the context is evaluated on measured histories, whereas during recursive prediction it is evaluated on predicted histories, so prediction errors alter the scheduling signal itself. Training therefore proceeds in two stages, which we treat as an optimization schedule rather than a requirement of the model: a nominal one-step initialization, followed by scheduled-rollout fine-tuning of the full history-conditioned family on the *propagated*-latent distribution that arises at deployment.

Stage 0 (nominal one-step pretraining). We initialize the encoder g_θ and a time-invariant base operator $(\mathbf{A}_0, \mathbf{B})$ by minimizing the one-step physical prediction error over the encoder-generated latent distribution,

$$\mathcal{L}_{\text{init}} = \sum_k \left\| \mathbf{C}\big(\mathbf{A}_0\, \mathbf{z}_k + \mathbf{B}\, \mathbf{u}_k\big) - \mathbf{x}_{k+1} \right\|_2^2, \qquad \mathbf{z}_k = g_\theta(\mathbf{X}_k). \tag{23}$$

This stage initializes the temporal encoder and the nominal input-affine predictor using one-step physical-state supervision; it is the only stage optimized exclusively on the encoder-consistent one-step latent distribution, and it corresponds to the fixed (time-invariant) Transformer-Koopman baseline. Because $\mathbf{C} = [\mathbf{I}_n \;\; \mathbf{0}]$ observes only the first n coordinates of the predicted latent, the loss (23) constrains the physical-output rows of $\mathbf{A}_0$ and $\mathbf{B}$; the remaining $d - n$ latent rows receive no gradient from this objective and are shaped by the initialization and by the rollout-aware optimization of Stage 1. Fig. 2 illustrates the one-step coupling used in this stage.

Definition 2 (Rollout-Aware Scheduled Objective)**.** *Starting from the encoder-initialized latent $\mathbf{z}_k^P = g_\theta(\mathbf{X}_k)$, the scheduled rollout is generated for $i = 0, \ldots, H_R - 1$ by*

$$\begin{aligned} \mathbf{c}_{k+i} &= C_\phi(\hat{\mathbf{X}}_{k+i}, \mathbf{U}_{k+i}^{\text{in}}), \\ \mathbf{z}_{k+i+1}^P &= \mathbf{A}(\mathbf{c}_{k+i})\, \mathbf{z}_{k+i}^P + \mathbf{B}\, \mathbf{u}_{k+i}, \\ \hat{\mathbf{x}}_{k+i+1} &= \mathbf{C}\, \mathbf{z}_{k+i+1}^P, \end{aligned} \tag{24}$$

where the windows are then shifted by appending the decoded state and the applied input and dropping the oldest entry, $\hat{\mathbf{X}}_{k+i+1} = \text{shift}(\hat{\mathbf{X}}_{k+i}, \hat{\mathbf{x}}_{k+i+1})$ and $\mathbf{U}_{k+i+1}^{\text{in}} = \text{shift}(\mathbf{U}_{k+i}^{\text{in}}, \mathbf{u}_{k+i})$. The lifting encoder is thus evaluated only at the start of the rollout, while the context is re-scheduled at every step from the history that the predictor itself has produced. With uniform weights $w_i = 1/H_R$, the scheduled objective is

$$\mathcal{L}_R = \sum_k \sum_{i=1}^{H_R} w_i \left\| \mathbf{C}\mathbf{z}_{k+i}^P - \mathbf{x}_{k+i} \right\|_2^2. \tag{25}$$

At deployment the transition consumes *propagated* latents $\mathbf{z}^P$, whose distribution differs from the encoder-generated one-step distribution on which Stage 0 is fitted. The rollout objective is an algorithmic training mechanism rather than a theoretical guarantee: no statement in Section V uses $\mathcal{L}_R$, and Section V analyzes rollout consistency independently through finite-horizon error propagation.

Stage 1 (scheduled-rollout fine-tuning). We then activate the low-rank context modulation $\mathbf{A}(\mathbf{c}_k) = \mathbf{A}_0 + \mathbf{U}\,\mathrm{diag}(\gamma_\psi(\mathbf{c}_k))\,\mathbf{V}^\top$ and jointly refine θ, ϕ, $\mathbf{A}_0$, $\mathbf{U}$, $\mathbf{V}$, γ_ψ, and the shared input matrix $\mathbf{B}$ by minimizing the rollout-aware scheduled objective $\mathcal{L}_R$ of Definition 2 (25), in which the rollout (24) propagates the latent and *re-schedules* the context from the history that the predictor itself has produced (Section V-A). Only the projection $\mathbf{C}$ is structurally fixed. The horizon and weights used in this work are reported with the experimental configuration.

Minimizing $\mathcal{L}_R$ trains the operator on the propagated-latent distribution that arises at deployment, rather than on the encoder-generated one-step distribution; this is the rollout mismatch between encoder-initialized and recursively propagated latent states analyzed in Section V-A.

The modulation is parameterized with a total absolute budget that limits the magnitude of the context-dependent correction: the columns of $\mathbf{U}$ and $\mathbf{V}$ are reparameterized to unit norm, and γ_ψ is reparameterized so that its components obey $\sum_{i=1}^{r}|\gamma_{\psi,i}(\mathbf{c})| \le g_{\max}$ for all $\mathbf{c}$, their signs being unconstrained. Both are structural reparameterizations rather than training penalties. Writing $\mathbf{A}(\mathbf{c}) - \mathbf{A}_0 = \sum_{i=1}^{r}\gamma_{\psi,i}(\mathbf{c})\,\mathbf{U}_{:,i}\mathbf{V}_{:,i}^\top$ with unit-norm columns $\|\mathbf{U}_{:,i}\|_2 = \|\mathbf{V}_{:,i}\|_2 = 1$, the triangle inequality together with $\|\mathbf{U}_{:,i}\mathbf{V}_{:,i}^\top\|_2 = 1$ gives

$$\|\mathbf{A}(\mathbf{c}) - \mathbf{A}_0\|_2 \le \sum_{i=1}^{r}\big|\gamma_{\psi,i}(\mathbf{c})\big| \le g_{\max}, \tag{26}$$

uniformly in $\mathbf{c}$. This bound controls the size of the correction only: it does not imply that $\mathbf{A}(\mathbf{c})$ is well-conditioned, that $\|\mathbf{A}(\mathbf{c})\|_2 < 1$ or $\rho(\mathbf{A}(\mathbf{c})) < 1$, or that the scheduled products $\mathbf{A}_{k+H-1}\cdots\mathbf{A}_k$ contract, and no stability property is claimed on this basis (Section V).

The complete procedure is summarized in Algorithm 1.

Algorithm 1 Transformer-Koopman Training Algorithm

Input: Trajectory dataset $\mathcal{D}$ (9)
History length P; patch length p; lifted dimension d; rank r; rollout horizon H_R
Rollout weights $w_1, \dots, w_{H_R}$; epochs E_0 (nominal pre-training), E_1 (scheduled-rollout fine-tuning)
Output: Lifting encoder θ^*; context ϕ^*; transition parameters $\mathbf{A}_0^*, \mathbf{U}^*, \mathbf{V}^*, \gamma_\psi^*$; input matrix $\mathbf{B}^*$
Stage 0 (nominal one-step pretraining): Initialize θ and $(\mathbf{A}_0, \mathbf{B})$; for E_0 epochs, minimize the one-step loss (23) over $(\theta, \mathbf{A}_0, \mathbf{B})$.
Stage 1 (scheduled-rollout fine-tuning): Activate $\mathbf{A}(\mathbf{c}) = \mathbf{A}_0 + \mathbf{U}\,\mathrm{diag}(\gamma_\psi(\mathbf{c}))\,\mathbf{V}^\top$ and initialize $(\phi, \mathbf{U}, \mathbf{V}, \psi)$; keep only $\mathbf{C}$ structurally fixed.
for epoch $\ell = 1$ **to** E_1 **do**
  for each mini-batch $\mathcal{S}$ of length-H_R segments drawn from $\mathcal{D}$ **do**
    Initialize the rollout at $\mathbf{z}_k^P = g_\theta(\mathbf{X}_k)$.
    for $i = 0$ **to** $H_R - 1$ **do**
      Re-schedule the context $\mathbf{c}_{k+i} = C_\phi(\hat{\mathbf{X}}_{k+i}, \mathbf{U}_{k+i}^{\mathrm{in}})$ from the rolled-out history.
      Propagate $\mathbf{z}_{k+i+1}^P = \mathbf{A}(\mathbf{c}_{k+i})\mathbf{z}_{k+i}^P + \mathbf{B}\mathbf{u}_{k+i}$ and shift the windows using $\hat{\mathbf{x}}_{k+i+1} = \mathbf{C}\mathbf{z}_{k+i+1}^P$.
    end for
    Form the scheduled objective $\mathcal{L}_R$ (25).
    Update $(\theta, \phi, \mathbf{A}_0, \mathbf{U}, \mathbf{V}, \gamma_\psi, \mathbf{B})$ by backpropagation through the rollout.
  end for
end for
return $\theta^*, \phi^*, \mathbf{A}_0^*, \mathbf{U}^*, \mathbf{V}^*, \gamma_\psi^*, \mathbf{B}^*$

V. Finite-Horizon Consistency Analysis

In this section, we analyze the internal finite-history predictor of Section IV and its finite-horizon error propagation. Section V-A bounds the inconsistency that arises when the latent state is propagated recursively instead of being re-encoded, and Section V-B bounds the finite-horizon prediction error of the learned augmented predictor against a recorded reference sequence. All statements are conditional on the regularity assumptions stated below and concern the predictor itself; the scope remark at the end of the section states what is *not* claimed.

Because the lifted state is a function of a finite window, the predictor can be recast as a first-order system on an augmented state. Define

$$\boldsymbol{\xi}_k = \mathrm{col}\big(\mathrm{vec}\,\mathbf{X}_k,\ \mathrm{vec}\,\mathbf{U}_k^{\mathrm{in}},\ \mathbf{z}_k\big) \in \mathbb{R}^{Pn+(P-1)m+d}, \tag{27}$$

and consider the predictor's own rollout, in which the newest fed-back state is its decoded output $\hat{\mathbf{x}}_{k+1} = \mathbf{C}\mathbf{z}_{k+1}$. The window update is then a deterministic shift: append the decoded state and the applied input, and drop the oldest entry. Since $\mathbf{X}_k$ and $\mathbf{U}_k^{\mathrm{in}}$ are sub-blocks of $\boldsymbol{\xi}_k$, every argument of that update is a function of $(\boldsymbol{\xi}_k, \mathbf{u}_k)$ alone, so composing the shift, the context map, the transition, and the decoder yields a single deterministic map

$$\boldsymbol{\xi}_{k+1} = \mathbf{F}_\Theta(\boldsymbol{\xi}_k, \mathbf{u}_k). \tag{28}$$

This construction describes the predictor's internal rollout only. If a true measurement $\mathbf{x}_{k+1}$ is fed back instead, the update also depends on an exogenous quantity and is not a function of $\boldsymbol{\xi}_k$ alone; and nothing here asserts that the finite measured history Markovizes the physical system (1).

A. Latent Rollout Inconsistency and Its Propagation

We now characterize the discrepancy that arises when the latent state is propagated recursively instead of being re-encoded at every step.

Definition 3 (Latent Rollout Inconsistency)**.** *Let $\hat{\mathbf{X}}_{k+i}$ be the history produced by the predictor's own rollout and let $\mathbf{c}_{k+i} = C_\phi(\hat{\mathbf{X}}_{k+i}, \mathbf{U}_{k+i}^{\mathrm{in}})$ denote the scheduled context re-*

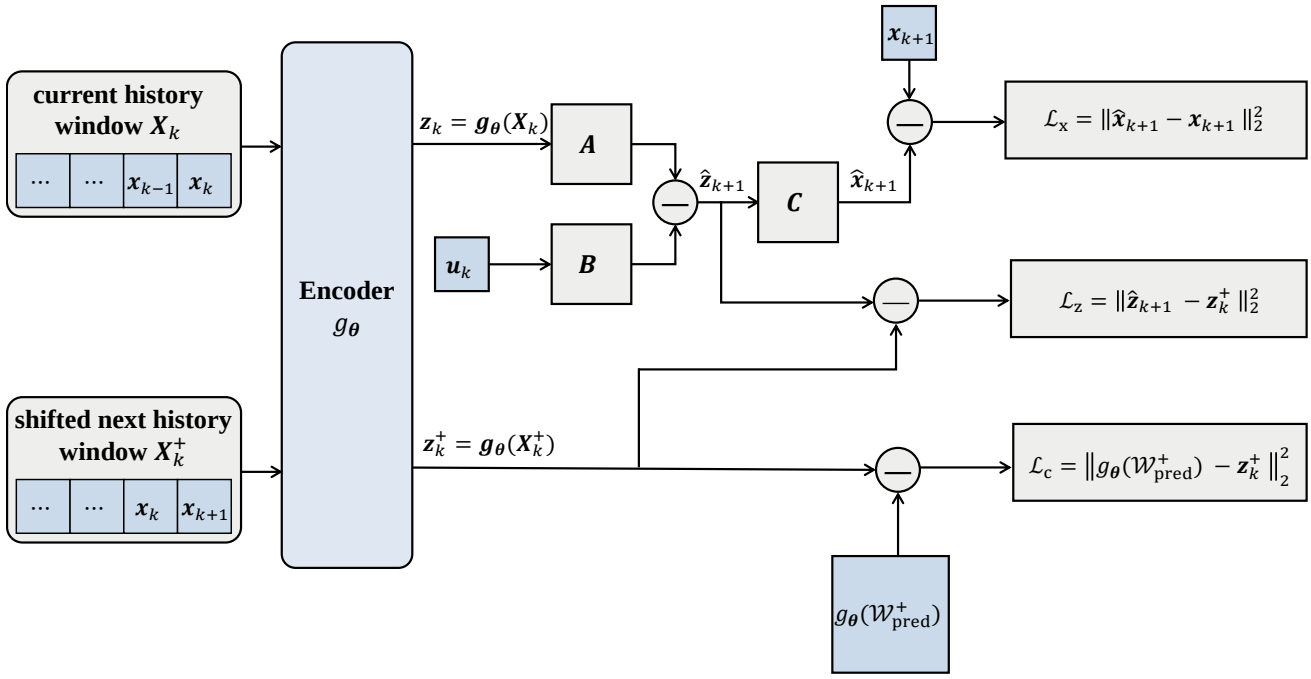


Fig. 2. One-step lifting and prediction (nominal pretraining, Stage 0). The history window $\mathbf{X}_k$ is encoded by g_θ to produce the lifted state $\mathbf{z}_k$, and the time-invariant base operator $(\mathbf{A}_0, \mathbf{B})$ produces the one-step prediction $\hat{\mathbf{z}}_{k+1} = \mathbf{A}_0\mathbf{z}_k + \mathbf{B}\mathbf{u}_k$, from which $\hat{\mathbf{x}}_{k+1}$ is recovered via $\mathbf{C}$. Stage-0 training is supervised only through the decoded physical state, by the one-step loss (23); the shifted window $\mathbf{X}_k^+$ indicates the sliding-window convention over the dataset and does not enter the Stage-0 objective. Stage 1 replaces $\mathbf{A}_0$ by the history-conditioned $\mathbf{A}(\mathbf{c})$ and jointly refines the full model on the rollout-aware scheduled objective (Algorithm 1).

computed from it. Define the encoder-consistent latent and the propagated latent,

$$\mathbf{z}_{k+i}^E = g_\theta(\hat{\mathbf{X}}_{k+i}), \qquad \mathbf{z}_{k+i+1}^P = \mathbf{A}(\mathbf{c}_{k+i})\mathbf{z}_{k+i}^P + \mathbf{B}\mathbf{u}_{k+i}, \tag{29}$$

and the normalized rollout defect

$$d_{\mathcal{M},k+i} = \frac{\left\|\mathbf{z}_{k+i}^P - \mathbf{z}_{k+i}^E\right\|_2}{\left\|\mathbf{z}_{k+i}^E\right\|_2 + \epsilon_0}, \tag{30}$$

with $\epsilon_0 = 10^{-6}$ a fixed numerical floor. We also use the absolute defect $d_{k+i}^{\mathrm{abs}} = \|\mathbf{z}_{k+i}^P - \mathbf{z}_{k+i}^E\|_2$, which is the quantity for which Proposition 1 provides an upper bound.

Two properties of this defect are worth stating explicitly. First, because $\mathbf{z} = [\mathbf{x}; \mathbf{h}]$ is state-preserving and the newest entry of $\hat{\mathbf{X}}_{k+i}$ is the decoded state $\mathbf{C}\mathbf{z}_{k+i}^P$, the propagated and the encoder-consistent latents share their first n coordinates, $\mathbf{C}(\mathbf{z}_{k+i}^P - \mathbf{z}_{k+i}^E) = \mathbf{0}$; the defect therefore measures the inconsistency of the remaining $d-n$ hidden lifted coordinates rather than a physical multi-step prediction error. Those hidden coordinates still affect future decoded predictions through $\mathbf{A}(\mathbf{c})$, which is why this inconsistency matters. Second, evaluating $\mathbf{z}_{k+i}^E$ requires an additional encoder pass on the predicted history. That pass is an offline diagnostic used to characterize the mechanism; it is not part of the online rollout, in which the context is re-scheduled without re-running the encoder inside the horizon (Section VI).

Proposition 1 (Latent Rollout Error Propagation)**.** *Let $\mathbf{z}_k^P$ denote the recursively propagated latent state and $\mathbf{z}_k^E = g_\theta(\hat{\mathbf{X}}_k)$ the encoder-consistent latent state, and let $d_k \equiv d_k^{\mathrm{abs}} = \|\mathbf{z}_k^P - \mathbf{z}_k^E\|_2$. Define the encoder-consistent one-step defect*

$$\epsilon_k = \left\|\mathbf{A}(\mathbf{c}_k)\mathbf{z}_k^E + \mathbf{B}\mathbf{u}_k - \mathbf{z}_{k+1}^E\right\|_2. \tag{31}$$

Then $d_{k+1} \le \|\mathbf{A}(\mathbf{c}_k)\|_2\, d_k + \epsilon_k$, and for any horizon H,

$$d_{k+H} \le \left\|\mathbf{A}_{k+H-1}\cdots\mathbf{A}_k\right\|_2 d_k + \sum_{i=0}^{H-1} \left\|\mathbf{A}_{k+H-1}\cdots\mathbf{A}_{k+i+1}\right\|_2 \epsilon_{k+i}, \tag{32}$$

where $\mathbf{A}_{k+j} = \mathbf{A}(\mathbf{c}_{k+j})$ is the transition at the scheduled context $\mathbf{c}_{k+j} = C_\phi(\hat{\mathbf{X}}_{k+j}, \mathbf{U}_{k+j}^{\mathrm{in}})$ along the same rollout. If the rollout starts from an encoder-consistent latent ($\mathbf{z}_k^P = \mathbf{z}_k^E$, hence $d_k = 0$), the first term vanishes and the bound reduces to the second sum alone. The bound is a trajectory-wise recursion: both ϵ_k and the transition matrices $\mathbf{A}_{k+j}$ are evaluated along the same rollout, so they depend on the realized trajectory rather than being a priori constants.

Proof. Let $\mathbf{e}_k = \mathbf{z}_k^P - \mathbf{z}_k^E$ denote the latent error. Subtracting the encoder-consistent update from the propagation rule $\mathbf{z}_{k+1}^P = \mathbf{A}(\mathbf{c}_k)\mathbf{z}_k^P + \mathbf{B}\mathbf{u}_k$ gives the linear recursion

$$\mathbf{e}_{k+1} = \mathbf{A}(\mathbf{c}_k)\mathbf{e}_k + \mathbf{r}_k^z, \qquad \mathbf{r}_k^z = \mathbf{A}(\mathbf{c}_k)\mathbf{z}_k^E + \mathbf{B}\mathbf{u}_k - \mathbf{z}_{k+1}^E,$$

where $\|\mathbf{r}_k^z\|_2 = \epsilon_k$. Taking norms and using the triangle inequality and submultiplicativity yields the one-step bound

$d_{k+1} \le \|\mathbf{A}(\mathbf{c}_k)\|_2\, d_k + \epsilon_k$. Unrolling the recursion over H steps gives

$$\mathbf{e}_{k+H} = \mathbf{A}_{k+H-1}\cdots\mathbf{A}_k\,\mathbf{e}_k + \sum_{i=0}^{H-1} \mathbf{A}_{k+H-1}\cdots\mathbf{A}_{k+i+1}\,\mathbf{r}^z_{k+i},$$

and taking norms, with $\|\mathbf{e}_k\|_2 = d_k$ and $\|\mathbf{r}^z_{k+i}\|_2 = \epsilon_{k+i}$, gives (32) (the empty product at $i = H-1$ is the identity). Since $\|\mathbf{A}_{k+H-1}\cdots\mathbf{A}_k\|_2 \le \prod_{j=0}^{H-1}\|\mathbf{A}_{k+j}\|_2$, this matrix-product form is at least as tight as the product-of-norms form. □

Equation (32) shows that the long-horizon inconsistency is governed by two factors: the local one-step defect ϵ_k and the *amplification of the time-varying transition products*.

Remark 1 (Why spectral stabilization is insufficient)**.** *Enforcing $\rho(\mathbf{A}(\mathbf{c}_k)) < 1$ for all k does* not *imply a uniform contraction of the time-varying products:*

$$\rho(\mathbf{A}_k) < 1 \;\;\forall k \quad\not\Rightarrow\quad \|\mathbf{A}_{k+H-1}\cdots\mathbf{A}_k\|_2 \le c < 1, \tag{33}$$

which is the classical spectral-radius/joint-spectral-radius gap for non-commuting matrices [40]–[42]. A bound on $\rho(\mathbf{A}(\mathbf{c}))$ therefore does not by itself control the amplification of the scheduled products that appears in (32)*.*

B. Finite-Horizon Prediction Error Bound

To state the bound we compare the learned predictor with a *reference augmented sequence* built directly from the recorded observations, rather than postulating a deterministic reference transition for the plant. Let

$$\boldsymbol{\xi}^\star_k = \mathrm{col}\big(\mathrm{vec}\,\mathbf{X}_k,\ \mathrm{vec}\,\mathbf{U}^{\mathrm{in}}_k,\ g_\theta(\mathbf{X}_k)\big) \tag{34}$$

denote the reference augmented state built from the true observations and the encoder-consistent latent, and let $\boldsymbol{\xi}^\star_{k+1}$ be its successor along the recorded trajectory. The reference sequence consists of measured quantities; no deterministic reference map is assumed to generate it, so the finite observed history is never asserted to Markovize the underlying plant. The learned augmented transition $\mathbf{F}_\Theta$ is the map (28), which propagates the latent and feeds back the decoded output $\hat{\mathbf{x}} = \mathbf{C}\mathbf{z}^P$. We therefore measure the learned model by its one-step discrepancy on the reference sequence,

$$\delta^\star_k = \big\|\boldsymbol{\xi}^\star_{k+1} - \mathbf{F}_\Theta(\boldsymbol{\xi}^\star_k, \mathbf{u}_k)\big\|_2. \tag{35}$$

This quantity bundles the physical modeling error, the finite-history truncation error, the learned transition error, and the difference between the encoder-consistent and the propagated latent updates.

On the recorded sequence we also define the one-step modeling residual at the encoder-consistent latent,

$$\mathbf{w}_k = \mathbf{z}^E_{k+1} - \big(\mathbf{A}(\mathbf{c}_k)\mathbf{z}^E_k + \mathbf{B}\mathbf{u}_k\big), \qquad \mathbf{z}^E_k = g_\theta(\mathbf{X}_k), \tag{36}$$

which measures one-step data consistency on recorded histories. It is not an independent error quantity: because the history and input shifts are exact and the encoder is state-preserving, $\mathbf{C}\mathbf{z}^E_{k+1} = \mathbf{x}_{k+1}$, and the augmented one-step reference discrepancy (35) reduces to

$$(\delta^\star_k)^2 = \|\mathbf{w}_k\|_2^2 + \|\mathbf{C}\mathbf{w}_k\|_2^2, \qquad \|\mathbf{w}_k\|_2 \le \delta^\star_k \le \sqrt{2}\,\|\mathbf{w}_k\|_2, \tag{37}$$

so $\delta^\star_k$ and $\mathbf{w}_k$ are two views of the same one-step discrepancy on recorded data. This identity uses the history shift, the fixed projection $\mathbf{C} = [\mathbf{I}_n\ \ \mathbf{0}]$, and the re-encoding of the next recorded state adopted here; it is not an unconditional relation for arbitrary feedback structures. The defect ϵ_k of Proposition 1, by contrast, is evaluated on histories generated by the predictor itself and cannot be replaced by statistics of $\mathbf{w}_k$.

Assumption 1 (Operating-set reference discrepancy)**.** *There exist a compact* operating set $\Omega \subset \mathbb{R}^{Pn+(P-1)m+d} \times \mathbb{R}^m$*, i.e. a set of augmented state–input pairs $(\boldsymbol{\xi}, \mathbf{u})$, and a finite constant $\epsilon \ge 0$ such that, along the reference sequence and at every step of the considered horizon, the one-step reference discrepancy* (35) *satisfies $\delta^\star_k \le \epsilon$, equivalently*

$$\big\|\boldsymbol{\xi}^\star_{k+1} - \mathbf{F}_\Theta(\boldsymbol{\xi}^\star_k, \mathbf{u}_k)\big\|_2 \le \epsilon. \tag{38}$$

We additionally require that the reference and predicted trajectories remain in Ω over the considered horizon; this condition is used explicitly in Theorem 1.

Assumption 2 (Lipschitz continuity of the learned transition)**.** *There exists a finite constant L_{F} such that*

$$\|\mathbf{F}_\Theta(\boldsymbol{\xi}_1, \mathbf{u}) - \mathbf{F}_\Theta(\boldsymbol{\xi}_2, \mathbf{u})\|_2 \le L_{\mathrm{F}}\|\boldsymbol{\xi}_1 - \boldsymbol{\xi}_2\|_2 \tag{39}$$

for all $(\boldsymbol{\xi}_1, \mathbf{u}), (\boldsymbol{\xi}_2, \mathbf{u}) \in \Omega$. No contraction is assumed: L_{F} need not be smaller than one. The bound holds on Ω whenever the context map C_ϕ, the modulation γ_ψ, and the map $\mathbf{c} \mapsto \mathbf{A}(\mathbf{c})$ are Lipschitz there and the lifted state stays bounded; we take these as standing regularity conditions on the learned components. These conditions concern the functions that the recursion (28) *actually composes—the context map, the modulation, the low-rank transition, the affine latent update and decoding, and the exact window shift—and not the lifting encoder: g_θ is evaluated only to initialize the rollout, and is not re-evaluated inside $\mathbf{F}_\Theta$. In the parameterization used here the context map and the modulation are composed of affine layers and smooth activations, so on a bounded operating set these conditions are met.*

Theorem 1 (Finite-Horizon Prediction Error Bound)**.** *For a common prescribed input sequence $\{\mathbf{u}_k, \dots, \mathbf{u}_{k+H-1}\}$, suppose that the reference and predicted trajectories remain in the operating set Ω over the horizon H. Under Assumptions 1 and 2, and with $e_k = \boldsymbol{\xi}^\star_k - \hat{\boldsymbol{\xi}}_k$,*

$$\|e_{k+H}\|_2 \le L_{\mathrm{F}}^H\|e_k\|_2 + \sum_{i=0}^{H-1} L_{\mathrm{F}}^{H-1-i}\,\delta^\star_{k+i}. \tag{40}$$

If in addition the reference discrepancy is uniformly bounded by ϵ as in Assumption 1, then

$$\|e_{k+H}\|_2 \le L_{\mathrm{F}}^H\|e_k\|_2 + \epsilon\sum_{j=0}^{H-1} L_{\mathrm{F}}^j. \tag{41}$$

If the horizon is initialized from the same measured history and the corresponding encoder-consistent latent state, i.e. $\hat{\mathbf{X}}_k = \mathbf{X}_k$ *and* $\mathbf{z}_k^P = g_\theta(\mathbf{X}_k)$*, then* $e_k = 0$ *and* $\|e_{k+H}\|_2 \le \epsilon \sum_{j=0}^{H-1} L_{\mathrm{F}}^j$*. The bound compares the learned predictor with the recorded reference trajectory under a* common *input sequence; it is not a closed-loop tracking-error or robustness statement. Here* ϵ *is the uniform bound of Assumption 1.*

Proof. For one step, using the recorded successor $\boldsymbol{\xi}_{k+1}^\star$,

$$\begin{aligned}\|e_{k+1}\|_2 &= \big\|\boldsymbol{\xi}_{k+1}^\star - \mathbf{F}_\Theta(\hat{\boldsymbol{\xi}}_k, \mathbf{u}_k)\big\|_2 \\ &\le \big\|\boldsymbol{\xi}_{k+1}^\star - \mathbf{F}_\Theta(\boldsymbol{\xi}_k^\star, \mathbf{u}_k)\big\|_2 \\ &\quad + \big\|\mathbf{F}_\Theta(\boldsymbol{\xi}_k^\star, \mathbf{u}_k) - \mathbf{F}_\Theta(\hat{\boldsymbol{\xi}}_k, \mathbf{u}_k)\big\|_2 \\ &\le \delta_k^\star + L_{\mathrm{F}}\|e_k\|_2,\end{aligned}$$

by (35) and Assumption 2, which applies at every step because the trajectories remain in Ω over the horizon. Unrolling the recursion yields (40); bounding each $\delta_{k+i}^\star$ by ϵ yields (41). □

Remark 2 (How to read these bounds)**.** *Equation* (40) *is a conditional finite-horizon relation between two trajectories driven by the same input sequence, not a numerical certificate. The quantities* ϵ*,* ϵ_k *and* $\mathbf{w}_k$ *are defined on different trajectories and are not interchangeable, and the regularity constants of Assumptions 1 and 2 are properties of the operating set* Ω *rather than of any validation sample. The bound concerns the rolled-out predictor under a common input sequence: when* $L_{\mathrm{F}} > 1$ *it grows geometrically with* H*, and it is neither a closed-loop tracking-error statement nor a stability result.*

The bound above concerns the augmented state, and the physical coordinates are a sub-vector of it. Both the current physical state $\mathbf{x}_{k+H}$ and the predictor's decoded feedback $\hat{\mathbf{x}}_{k+H} = \mathbf{C}\mathbf{z}_{k+H}^P$ occupy the last n coordinates of the history block of the respective augmented states, so their difference is a coordinate sub-vector of $\hat{\boldsymbol{\xi}}_{k+H} - \boldsymbol{\xi}_{k+H}^\star$. Since a coordinate sub-vector has at most the norm of the full vector, Theorem 1 gives directly, with no additional assumption,

$$\|\hat{\mathbf{x}}_{k+H} - \mathbf{x}_{k+H}\|_2 \le L_{\mathrm{F}}^H \|e_k\|_2 + \sum_{i=0}^{H-1} L_{\mathrm{F}}^{H-1-i}\, \delta_{k+i}^\star. \quad (42)$$

Remark 3 (Scope of the analysis)**.** *The analysis is finite-horizon and concerns the consistency of the learned predictor and the propagation of its prediction error; the local linearization accuracy of the RTI step is treated separately in Section VI. It does not establish global closed-loop stability, a common Lyapunov function, terminal invariant sets, or recursive feasibility under learned uncertainty. The regularity constants used for the empirical assessment are operating-region diagnostics rather than global certificates, all bounds concern the rolled-out latent predictor rather than true-plant tracking error, and* $\mathbf{A}(\mathbf{c})$ *is a latent linearization parameterization rather than an exact Koopman invariant-subspace operator.*

VI. Real-Time Predictive Control

In this section, we embed the learned history-conditioned predictor into a receding-horizon controller. The prediction rollout propagates the lifted state while updating the scheduling context from decoded state histories, and the optimization is approximated by one real-time-iteration (RTI) step per control update [43], [44] to reduce the online computation of each update; whether a given sampling constraint is met is an empirical question.

A. History-Consistent Scheduled Prediction

At control instant k the controller is initialized from the measured history and predicts over a horizon N according to

$$\begin{aligned}\hat{\mathbf{X}}_{k|k} &= \mathbf{X}_k, \qquad \mathbf{U}_{k|k}^{\mathrm{in}} = \mathbf{U}_k^{\mathrm{in}}, \\ \mathbf{z}_{k|k} &= g_\theta(\mathbf{X}_k), \\ \mathbf{c}_{k+i|k} &= C_\phi\big(\hat{\mathbf{X}}_{k+i|k}, \mathbf{U}_{k+i|k}^{\mathrm{in}}\big), \\ \mathbf{z}_{k+i+1|k} &= \mathbf{A}(\mathbf{c}_{k+i|k})\, \mathbf{z}_{k+i|k} + \mathbf{B}\, \mathbf{u}_{k+i|k}, \\ \hat{\mathbf{x}}_{k+i+1|k} &= \mathbf{C}\, \mathbf{z}_{k+i+1|k}, \\ \hat{\mathbf{X}}_{k+i+1|k} &= \mathrm{shift}\big(\hat{\mathbf{X}}_{k+i|k}, \hat{\mathbf{x}}_{k+i+1|k}\big), \\ \mathbf{U}_{k+i+1|k}^{\mathrm{in}} &= \mathrm{shift}\big(\mathbf{U}_{k+i|k}^{\mathrm{in}}, \mathbf{u}_{k+i|k}\big), \end{aligned} \quad (43)$$

where $i = 0, \dots, N-1$ and $i|k$ denotes prediction step i from control instant k. This is the same *context-re-scheduled, latent-propagated* recursion as the training rollout (24): the lifting encoder is evaluated once per control instant, at the start of the horizon, and the context is re-scheduled at every step from the decoded history rather than re-encoded. At the next sampling instant the controller is re-initialized from the newly measured history, which does not re-run the encoder inside the horizon. The alternative—re-encoding the history at every prediction stage (RTI-RE)—requires one additional encoder evaluation per prediction stage; whether that additional evaluation improves accuracy is an empirical question left to the experiments.

B. One-Iteration RTI

Writing $\mathbf{U}_{\mathrm{s}} = \mathrm{col}(\mathbf{u}_{k|k}, \dots, \mathbf{u}_{k+N-1|k}) \in \mathbb{R}^{Nm}$ for the stacked input sequence over the horizon, we solve the receding-horizon problem

$$\begin{aligned}\min_{\mathbf{U}_{\mathrm{s}}} \quad & \sum_{i=1}^{N} \big\|\hat{\mathbf{x}}_{k+i|k} - \mathbf{x}_{k+i}^{\mathrm{ref}}\big\|_{\mathbf{Q}}^2 \\ & + \sum_{i=0}^{N-1} \Big(\big\|\mathbf{u}_{k+i|k}\big\|_{\mathbf{R}}^2 + \big\|\mathbf{u}_{k+i|k} - \mathbf{u}_{k+i-1|k}\big\|_{\mathbf{S}}^2\Big),\end{aligned} \quad (44)$$

subject to the componentwise input box $\mathbf{u}_{\min} \le \mathbf{u}_{k+i|k} \le \mathbf{u}_{\max}$. In the input-rate term the predecessor is the previously *applied* input, $\mathbf{u}_{k-1|k} = \mathbf{u}_{k-1}^{\mathrm{app}}$, which is a known constant rather than a decision variable, and $\mathbf{Q}, \mathbf{R}, \mathbf{S}$ are positive semidefinite weights. Because the scheduled context depends on the predicted history, the predicted states in (44) depend on the whole input sequence, so this problem is generally nonlinear and is not solved as a convex program; it is handled locally, as follows. No terminal cost, terminal set, state constraint, or robust tube is imposed, and none is claimed below.

One RTI step per sampling instant for (44) consists of: (i) warm-starting the input sequence from the previous solution shifted by one step; (ii) rolling out the nominal trajectory (43)

at that sequence; (iii) evaluating, along the nominal trajectory, the sensitivities

$$\begin{aligned} \mathbf{F}_i &= \frac{\partial \mathbf{F}_\Theta}{\partial \boldsymbol{\xi}}\Big|_{(\bar{\boldsymbol{\xi}}_i, \bar{\mathbf{u}}_i)}, \qquad \mathbf{G}_i = \frac{\partial \mathbf{F}_\Theta}{\partial \mathbf{u}}\Big|_{(\bar{\boldsymbol{\xi}}_i, \bar{\mathbf{u}}_i)}, \\ \delta\boldsymbol{\xi}_{i+1} &= \mathbf{F}_i\, \delta\boldsymbol{\xi}_i + \mathbf{G}_i\, \delta\mathbf{u}_i, \end{aligned} \tag{45}$$

and composing them along the rollout into the sensitivity of the predicted outputs to the *whole* input sequence; this retains the dependence of every scheduled context on the inputs applied before it, and does not treat $\mathbf{A}(\mathbf{c}_i)$ as a fixed matrix; (iv) solving one local quadratic subproblem in the increments $\delta\mathbf{U}_\mathrm{s}$ around the nominal value $\bar{\mathbf{U}}_\mathrm{s}$,

$$\begin{aligned} \min_{\delta\mathbf{U}_\mathrm{s}} \quad & \tfrac{1}{2}\, \delta\mathbf{U}_\mathrm{s}^\top \mathbf{H}_\mathrm{RTI}\, \delta\mathbf{U}_\mathrm{s} + \mathbf{g}_\mathrm{RTI}^\top \delta\mathbf{U}_\mathrm{s}, \\ \text{s.t.} \quad & \mathbf{u}_\mathrm{min} - \bar{\mathbf{U}}_\mathrm{s} \le \delta\mathbf{U}_\mathrm{s} \le \mathbf{u}_\mathrm{max} - \bar{\mathbf{U}}_\mathrm{s}, \end{aligned} \tag{46}$$

in which $\mathbf{H}_\mathrm{RTI}$ and $\mathbf{g}_\mathrm{RTI}$ are determined by the weights $\mathbf{Q}, \mathbf{R}, \mathbf{S}$, the nominal prediction, and the composed sensitivities of step (iii); $\mathbf{H}_\mathrm{RTI}$ is symmetric and positive semidefinite for the stated weights but need not be positive definite, and the subproblem is solved numerically rather than in closed form; and (v) applying only the first input, $\mathbf{u}_k^\mathrm{app} = \bar{\mathbf{u}}_k + \delta\mathbf{u}_k$.

Because the bounds act on the increments, $\delta\mathbf{u} = \mathbf{0}$ is feasible for this subproblem whenever the warm-start nominal input lies inside the box. This is feasibility of the linearized subproblem only: it does not imply recursive feasibility of the nonlinear problem, and no claim is made here of strict convexity, of a unique minimizer, of exact minimization, or of global SQP convergence. The practical adequacy of a single iteration is assessed empirically against a matched exact scheduled NMPC oracle in Section VII.

The accuracy of step (iii) is characterized by the local second-order remainder of the augmented map. Treating $\mathbf{F}_\Theta$ as a function of the joint variable $\eta = \mathrm{col}(\boldsymbol{\xi}, \mathbf{u})$, let $\bar{\eta}_i = \mathrm{col}(\bar{\boldsymbol{\xi}}_i, \bar{\mathbf{u}}_i)$ and $\delta\eta_i = \mathrm{col}(\delta\boldsymbol{\xi}_i, \delta\mathbf{u}_i)$ denote the nominal point and the joint perturbation.

Assumption 3 (Local second-order regularity of the augmented map)**.** *The nominal point $\bar{\eta}_i$ and the perturbation segment $\{\bar{\eta}_i + s\,\delta\eta_i : \ s \in [0,1]\}$ lie in a common region on which $\mathbf{F}_\Theta$ is twice continuously differentiable, and on that segment the second derivative is bounded by $\|\mathrm{D}^2\mathbf{F}_\Theta\|_2 \le M_\mathrm{F}$, where $\|\cdot\|_2$ denotes the operator norm of the second-derivative bilinear form. The remainder below is used only for perturbations whose segment lies within one such region: it is a local statement, and the same estimate is not asserted for a segment that crosses a boundary on which $\mathbf{F}_\Theta$ is not twice differentiable.*

Under Assumption 3 the expansion

$$\begin{aligned} \mathbf{F}_\Theta(\bar{\eta}_i + \delta\eta_i) &= \mathbf{F}_\Theta(\bar{\eta}_i) + \mathrm{D}\mathbf{F}_\Theta(\bar{\eta}_i)\, \delta\eta_i + \mathcal{R}_i, \\ \mathrm{D}\mathbf{F}_\Theta(\bar{\eta}_i)\, \delta\eta_i &= \mathbf{F}_i\, \delta\boldsymbol{\xi}_i + \mathbf{G}_i\, \delta\mathbf{u}_i, \end{aligned} \tag{47}$$

has the integral remainder

$$\mathcal{R}_i = \int_0^1 (1-s)\, \mathrm{D}^2\mathbf{F}_\Theta\big(\bar{\eta}_i + s\delta\eta_i\big)\big[\delta\eta_i, \delta\eta_i\big]\, \mathrm{d}s, \tag{48}$$

where $\mathrm{D}^2\mathbf{F}_\Theta[\cdot,\cdot]$ is the second-derivative bilinear form, and bounding the norm of the integral by the integral of the norm gives

$$\|\mathcal{R}_i\|_2 \le \frac{M_\mathrm{F}}{2}\big\|\delta\eta_i\big\|_2^2. \tag{49}$$

This bounds the model error of the local linearization used in step (iii), and it is local to the region of Assumption 3: it is a property of the linearization itself, and it neither bounds the performance of a single RTI iteration nor establishes SQP convergence, recursive feasibility, or closed-loop stability.

TABLE I
TRAINING-SCHEDULE ABLATION: VALIDATION ROLLOUT RMSE UNDER SCHEDULED PROPAGATION, AVERAGED OVER THREE SEEDS ($H_R = 20$ DURING TRAINING). THE STANDARD DEVIATION AT $H = 20$ IS AT MOST 0.0011 FOR B–D.

	Scheduled-rollout stage	$H=1$	$H=5$	$H=10$	$H=20$
A	modulation only	0.0276	0.1787	0.4300	1.1533
B	all but $\mathbf{B}$	0.0173	0.0271	0.0337	0.0440
C	all parameters	0.0115	0.0228	0.0301	0.0415
D	all, no pretraining	0.0112	0.0217	0.0289	0.0401

VII. EXPERIMENTS

This section reports a training-schedule ablation of the history-conditioned predictor, followed by the empirical diagnostics that assess the assumptions and quantities of the consistency analysis of Section V.

A. Training Schedule Ablation

We assess how the training schedule affects the deployment accuracy of the history-conditioned predictor. All variants share the same architecture, data, optimizer, rollout horizon ($H_R = 20$), learning rate, batch size, and scheduled-rollout-stage epoch budget. They differ in which parameters the scheduled-rollout stage optimizes: strategy A optimizes only the low-rank modulation $(\phi, \mathbf{U}, \mathbf{V}, \gamma_\psi)$ while freezing the temporal lifting θ, the nominal transition $\mathbf{A}_0$, and the input matrix $\mathbf{B}$; strategy B freezes only $\mathbf{B}$; strategy C optimizes all parameters; strategy D additionally omits the nominal one-step pretraining stage and optimizes all parameters from a random initialization. Table I reports the validation rollout RMSE under the deployment-consistent scheduled propagation, averaged over three seeds.

Three observations follow. First, what matters is which parameters the scheduled-rollout objective is allowed to adapt: optimizing only the low-rank modulation while freezing the temporal lifting and the nominal transition (A) degrades the $H = 20$ rollout error by more than an order of magnitude (1.1533 versus 0.0415), so the representation and the nominal transition must adapt together with the modulation. Second, freezing the input matrix brings no benefit: variant B is worse than variant C at every horizon and in every seed, with a 5.9% higher $H = 20$ rollout RMSE and a 24% higher rollout MSE accumulated over the whole 20-step rollout. Third, the nominal pretraining stage is not required: optimizing all parameters directly under the scheduled-rollout objective (D) matches or

TABLE II
THEORY DIAGNOSTICS ON THE FROZEN SCHEDULED-PROPAGATION PREDICTOR.

Quantity	Role	Measured ($H = 1/5/10/20$)
ϵ_k (absolute)	Prop. 1	median 0.345; q95 0.567
$d^{\mathrm{abs}}(H)$	Prop. 1	0.346/1.122/1.645/2.663
$d_{\mathcal{M}}(H)$	Def. 3	0.120/0.390/0.574/0.928
$\|\mathbf{A}_{k+H-1}\cdots\mathbf{A}_k\|_2$	Prop. 1	1.046/1.247/1.541/2.304
$\|\partial\mathbf{F}_\Theta/\partial\boldsymbol{\xi}\|_2$ ($\widehat{L_{\mathrm{F}}}$)	Assumption 2	median 1.445; max 1.452
Prop. 1 bound / actual	sanity	1.64/2.77/4.22/6.52×
$\widehat{M_{\mathrm{F}}}$ (fitted, δ^2-scaled)	(49)	≈ 0.007
physical RMSE	(42)	0.011/0.022/0.030/0.042
RTI runtime	deployment	p50 78 / p95 120 ms

slightly exceeds C while removing one training stage and reducing the total training time (311 s versus 338 s). We therefore treat the two-stage schedule of Section IV as an optimization initialization rather than a necessity of the model. All remaining results use the deployment predictor described in Section IV, which was trained with the same schedule as variant C; the ablation variants isolate the contribution of each element of that schedule. The separation of B from C and D exceeds the seed-to-seed standard deviation, whereas C and D are statistically indistinguishable.

We also verify that the deployment failure is not an artifact of unbounded latent dynamics. Training with an explicit spectral constraint $\rho(\mathbf{A}(\mathbf{c})) \leq 0.99$ under the one-step objective attains an $H = 20$ rollout RMSE of 1.5681, whereas the same constrained model trained under the scheduled-rollout objective attains 0.0412 (both single-seed comparisons). Enforcing spectral stability therefore does not repair the failure, whereas changing the training objective does.

B. Mechanism and Theory Diagnostics

Table II reports the diagnostics measured on the frozen deployment predictor of Section IV (all parameters optimized under the scheduled-rollout objective, $r = 2$, scheduled propagation) on the soft-robot validation set ($P = 8$, $n = 2$, $m = 2$, $d = 10$). These diagnostics are empirical measurements on the held-out validation distribution: they assess the consistency of the theoretical assumptions and quantities with the operating data, rather than certify global worst-case constants over the entire operating set Ω.[1]

Three observations follow, illustrated in Fig. 3. First, the transition-product norm grows from 1.05 to 2.30 (no contraction) in step with $d_{\mathcal{M}}$, numerically supporting Remark 1. Second, the empirical Lipschitz constant is $\widehat{L_{\mathrm{F}}} \approx 1.45 > 1$, so the finite-horizon form of Theorem 1 is the applicable statement and the infinite-horizon limit $\epsilon/(1-L_{\mathrm{F}})$ is not available. Third, the Proposition 1 bound is always $\geq$ the actual defect and only 1.64–$6.52\times$ conservative when compared like-for-like (absolute one-step defects ϵ_k against absolute rollout defects d^{abs}), and the fitted $\widehat{M_{\mathrm{F}}} \approx 0.007$ is stable across perturbation scales, supporting (49). The physical-space bound

[1] ϵ_k and d^{abs} are evaluated on all admissible validation windows (stride one); the H-step rollout statistics $d_{\mathcal{M}}$ and the transition-product norms use a stride-four subsample of the same trajectories. All values are medians unless a quantile is stated.

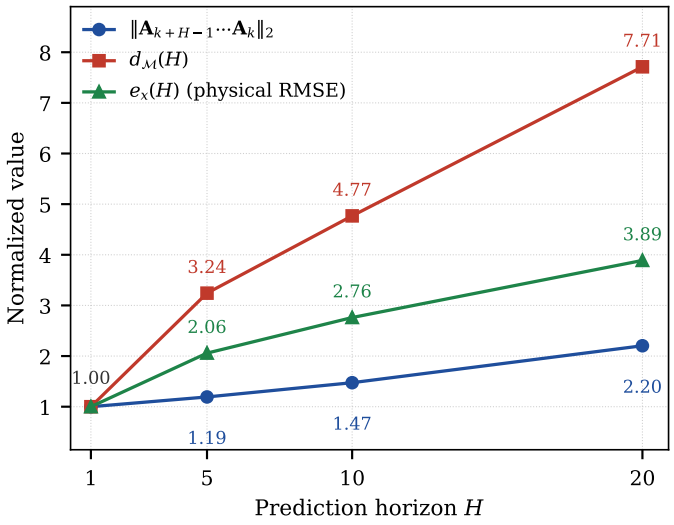


Fig. 3. Normalized growth with the prediction horizon H of the finite transition-product norm $\|\mathbf{A}_{k+H-1}\cdots\mathbf{A}_k\|_2$, the latent rollout defect $d_{\mathcal{M}}(H)$, and the physical prediction error $e_x(H)$; each curve is divided by its value at $H = 1$ so that different units share one axis. The three quantities grow together with H, which is *consistent with* the error-propagation mechanism of Proposition 1. The figure is an empirical illustration and does not by itself establish a causal chain.

(42) is valid but very loose (using $\widehat{L_{\mathrm{F}}} = 1.445$, 53–47 750× over $H = 1, \ldots, 20$), because $\widehat{L_{\mathrm{F}}}^{H}$ grows geometrically while the true error saturates; we report this honestly rather than presenting the bound as tight.

These measurements bear on the claim that one-step training insufficiently constrains the rollout distribution. That claim rests on the *conjunction* of four observations—a positive one-step consistency defect ($\epsilon_k > 0$), a rollout defect $d_{\mathcal{M}}(H)$ that grows with the horizon, the failure of naive latent propagation, and the recovery obtained under rollout-aware training—and no single observation is sufficient on its own; the latter two are quantified in the experimental study.

VIII. Conclusion

This paper presented Transformer-Koopman, a framework for learning history-conditioned Koopman models of controlled nonlinear systems from finite history windows. Because the measured state need not reveal the system's internal dynamics, a patch-based Transformer encoder first maps the history to a structured, state-preserving lifted state. Because a single global lifted transition would then still require all history-dependent regimes to share one evolution law, the lifted transition is additionally modulated by the history through a rank-r term $\mathbf{A}(\mathbf{c}) = \mathbf{A}_0 + \mathbf{U}\,\mathrm{diag}(\gamma_\psi(\mathbf{c}))\mathbf{V}^\top$ ($r = 2$), which confines history dependence to a structured correction of a shared nominal operator while keeping the propagation conditional-linear. We identified a training–deployment inconsistency between one-step-trained history-conditioned dynamics and the recursively generated latent states seen at deployment, showed that it is not repaired by spectral stabilization, and resolved it with deployment-consistent scheduled training. The rollout-trained scheduled predictor was integrated into a one-iteration RTI-MPC that

runs in real time and matches a matched exact scheduled NMPC oracle to within about 1%. A finite-horizon theory—finite-history Markovization, latent rollout error propagation, a bounded prediction-error theorem, a physical-space corollary, and a local RTI approximation result—was established and supported by empirical diagnostics. Hardware validation on the soft-robot platform across multi-setpoint, circular, figure-eight, unseen-trajectory, and speed-shift regimes, together with closed-loop baselines against a fixed (time-invariant) Transformer-Koopman controller, remains the immediate next step.

REFERENCES

[1] B. O. Koopman, "Hamiltonian systems and transformation in Hilbert space," *Proc. Natl. Acad. Sci. USA*, vol. 17, no. 5, pp. 315–318, 1931.

[2] B. O. Koopman and J. von Neumann, "Dynamical systems of continuous spectra," *Proc. Natl. Acad. Sci. USA*, vol. 18, no. 3, pp. 255–263, 1932.

[3] I. Mezić, "Spectral properties of dynamical systems, model reduction and decompositions," *Nonlinear Dyn.*, vol. 41, no. 1–3, pp. 309–325, 2005.

[4] L. Ljung, *System Identification: Theory for the User*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall PTR, 1999.

[5] P. Van Overschee and B. De Moor, "N4SID: Subspace algorithms for the identification of combined deterministic-stochastic systems," *Automatica*, vol. 30, no. 1, pp. 75–93, 1994.

[6] O. Nelles, *Nonlinear System Identification: From Classical Approaches to Neural Networks, Fuzzy Models, and Gaussian Processes*, 2nd ed. Cham, Switzerland: Springer, 2020.

[7] C. W. Rowley, I. Mezić, S. Bagheri, P. Schlatter, and D. S. Henningson, "Spectral analysis of nonlinear flows," *J. Fluid Mech.*, vol. 641, pp. 115–127, 2009.

[8] M. O. Williams, I. G. Kevrekidis, and C. W. Rowley, "A data–driven approximation of the Koopman operator: Extending dynamic mode decomposition," *J. Nonlinear Sci.*, vol. 25, no. 6, pp. 1307–1346, 2015.

[9] M. O. Williams, C. W. Rowley, and I. G. Kevrekidis, "A kernel-based method for data-driven Koopman spectral analysis," *J. Comput. Dyn.*, vol. 2, no. 2, pp. 247–265, 2015.

[10] H. Arbabi and I. Mezić, "Ergodic theory, dynamic mode decomposition, and computation of spectral properties of the Koopman operator," *SIAM J. Appl. Dyn. Syst.*, vol. 16, no. 4, pp. 2096–2126, 2017.

[11] M. Kamb, E. Kaiser, S. L. Brunton, and J. N. Kutz, "Time-delay observables for Koopman: Theory and applications," *SIAM J. Appl. Dyn. Syst.*, vol. 19, no. 2, pp. 886–917, 2020.

[12] S. L. Brunton, B. W. Brunton, J. L. Proctor, E. Kaiser, and J. N. Kutz, "Chaos as an intermittently forced linear system," *Nat. Commun.*, vol. 8, art. 19, 2017.

[13] S. Das and D. Giannakis, "Delay-coordinate maps and the spectra of Koopman operators," *J. Stat. Phys.*, vol. 175, no. 6, pp. 1107–1145, 2019.

[14] Y. Liu, C. Li, J. Wang, and M. Long, "Koopa: Learning non-stationary time series dynamics with Koopman predictors," in *Proc. NeurIPS*, 2023.

[15] Z. Li, M. Han, and X. Yin, "MamKO: Mamba-based Koopman operator for modeling and predictive control," in *Proc. ICLR*, 2025.

[16] S. L. Brunton, M. Budišić, E. Kaiser, and J. N. Kutz, "Modern Koopman theory for dynamical systems," *SIAM Rev.*, vol. 64, no. 2, pp. 229–340, 2022.

[17] S. E. Otto and C. W. Rowley, "Koopman operators for estimation and control of dynamical systems," *Annu. Rev. Control Robot. Auton. Syst.*, vol. 4, no. 1, pp. 59–87, 2021.

[18] N. Takeishi, Y. Kawahara, and T. Yairi, "Learning Koopman invariant subspaces for dynamic mode decomposition," in *Proc. NeurIPS*, 2017, pp. 1130–1140.

[19] E. Yeung, S. Kundu, and N. Hodas, "Learning deep neural network representations for Koopman operators of nonlinear dynamical systems," in *Proc. ACC*, 2019, pp. 4832–4839.

[20] B. Lusch, J. N. Kutz, and S. L. Brunton, "Deep learning for universal linear embeddings of nonlinear dynamics," *Nat. Commun.*, vol. 9, no. 1, p. 4950, 2018.

[21] J. Morton, F. D. Witherden, and M. J. Kochenderfer, "Deep variational Koopman models: Inferring Koopman observations for uncertainty-aware dynamics modeling and control," in *Proc. IJCAI*, 2019, pp. 3173–3179.

[22] O. Azencot, N. B. Erichson, V. Lin, and M. W. Mahoney, "Forecasting sequential data using consistent Koopman autoencoders," in *Proc. ICML*, 2020, pp. 475–485.

[23] L. Wu, W. G. Y. Tan, L. Zhou, R. D. Braatz, and J. Drgoňa, "Least-squares multi-step Koopman operator learning for model predictive control," arXiv:2601.11901, 2026.

[24] S. E. Otto and C. W. Rowley, "Linearly-recurrent autoencoder networks for learning dynamics," *SIAM J. Appl. Dyn. Syst.*, vol. 18, no. 1, pp. 558–593, 2019.

[25] M. Forgione and D. Piga, "dynoNet: A neural network architecture for learning dynamical systems," *Int. J. Adapt. Control Signal Process.*, vol. 35, no. 4, pp. 612–626, 2021.

[26] J. L. Proctor, S. L. Brunton, and J. N. Kutz, "Dynamic mode decomposition with control," *SIAM J. Appl. Dyn. Syst.*, vol. 15, no. 1, pp. 142–161, 2016.

[27] J. L. Proctor, S. L. Brunton, and J. N. Kutz, "Generalizing Koopman theory to allow for inputs and control," *SIAM J. Appl. Dyn. Syst.*, vol. 17, no. 1, pp. 909–930, 2018.

[28] S. L. Brunton, B. W. Brunton, J. L. Proctor, and J. N. Kutz, "Koopman invariant subspaces and finite linear representations of nonlinear dynamical systems for control," *PLOS ONE*, vol. 11, no. 2, e0150171, 2016.

[29] M. Korda and I. Mezić, "Linear predictors for nonlinear dynamical systems: Koopman operator meets model predictive control," *Automatica*, vol. 93, pp. 149–160, 2018.

[30] Y. Han, W. Hao, and U. Vaidya, "Deep learning of Koopman representation for control," in *Proc. IEEE Conf. Decision Control (CDC)*, 2020, pp. 1890–1895.

[31] H. Shi and M. Q.-H. Meng, "Deep Koopman operator with control for nonlinear systems," *IEEE Robot. Autom. Lett.*, vol. 7, no. 3, pp. 7700–7707, 2022.

[32] R. Strässer, K. Worthmann, I. Mezić, J. Berberich, M. Schaller, and F. Allgöwer, "An overview of Koopman-based control: From error bounds to closed-loop guarantees," *Annu. Rev. Control*, vol. 61, art. 101035, 2026.

[33] J. F. Durán-Sigüenza, L. I. Minchala, L. E. Garza-Castañón, and H. Zhang, "Control based on the Koopman operator: A comprehensive review," *J. Franklin Inst.*, vol. 362, no. 18, art. 108256, 2025.

[34] S. Klus, F. Nüske, P. Koltai, H. Wu, I. Kevrekidis, C. Schütte, and F. Noé, "Data-driven model reduction and transfer operator approximation," *J. Nonlinear Sci.*, vol. 28, no. 3, pp. 985–1010, 2018.

[35] A. Vaswani *et al.*, "Attention is all you need," in *Proc. NeurIPS*, 2017, pp. 5998–6008.

[36] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with Transformers," in *Proc. ICLR*, 2023.

[37] A. Forootani, M. Khosravi, and M. Barati, "DeepKoopFormer: A Koopman enhanced Transformer based architecture for time series forecasting," arXiv:2508.02616, 2025.

[38] A. Forootani and R. Iervolino, "Learnable Koopman-enhanced Transformer-based time series forecasting with spectral control," arXiv:2602.02592, 2026.

[39] N. Feizi, F. C. Pedrosa, J. Jayender, and R. V. Patel, "Deep Koopman approach for nonlinear dynamics and control of tendon-driven continuum robots," *IEEE Robot. Autom. Lett.*, vol. 10, no. 3, pp. 2926–2933, 2025.

[40] G.-C. Rota and G. Strang, "A note on the joint spectral radius," *Nederl. Akad. Wetensch. Proc. Ser. A*, vol. 63 (Indag. Math., vol. 22), pp. 379–381, 1960.

[41] I. Daubechies and J. C. Lagarias, "Sets of matrices all infinite products of which converge," *Linear Algebra Appl.*, vol. 161, pp. 227–263, 1992.

[42] R. M. Jungers, *The Joint Spectral Radius: Theory and Applications*, Lecture Notes in Control and Information Sciences, vol. 385. Berlin, Germany: Springer, 2009.

[43] M. Diehl, H. G. Bock, and J. P. Schlöder, "A real-time iteration scheme for nonlinear optimization in optimal feedback control," *SIAM J. Control Optim.*, vol. 43, no. 5, pp. 1714–1736, 2005.

[44] S. Gros, M. Zanon, R. Quirynen, A. Bemporad, and M. Diehl, "From linear to nonlinear MPC: Bridging the gap via the real-time iteration," *Int. J. Control*, vol. 93, no. 1, pp. 62–80, 2020.

[45] M. Diehl, R. Findeisen, F. Allgöwer, H. G. Bock, and J. P. Schlöder, "Nominal stability of real-time iteration scheme for nonlinear model predictive control," *IEE Proc. Control Theory Appl.*, vol. 152, no. 3, pp. 296–308, 2005.

[46] V. M. Zavala and L. T. Biegler, "The advanced-step NMPC controller: Optimality, stability and robustness," *Automatica*, vol. 45, no. 1, pp. 86–93, 2009.

[47] W. J. Rugh and J. S. Shamma, "Research on gain scheduling," *Automatica*, vol. 36, no. 10, pp. 1401–1425, 2000.
[48] D. J. Leith and W. E. Leithead, "Gain-scheduled and nonlinear systems: Dynamic analysis by velocity-based linearization families," *Int. J. Control*, vol. 70, no. 2, pp. 289–317, 1998.
[49] P. S. G. Cisneros, S. Voss, and H. Werner, "Efficient nonlinear model predictive control via quasi-LPV representation," in *Proc. 55th IEEE Conf. Decision Control (CDC)*, 2016, pp. 3216–3221.
[50] P. S. G. Cisneros and H. Werner, "Nonlinear model predictive control for models in quasi-linear parameter varying form," *Int. J. Robust Nonlinear Control*, vol. 30, no. 10, pp. 3945–3959, 2020.